Fault-tolerant Gang Scheduling in Distributed Real-time Systems Utilizing Imprecise Computations

Georgios L. Stavrinides
Helen D. Karatza
Department of Informatics
Aristotle University of Thessaloniki
54124 Thessaloniki, Greece
{gstavrin, karatza}@csd.auth.gr

Distributed real-time systems play an increasingly vital role in our society. The most important aspect of such systems is the scheduling algorithm, which must guarantee that every job in the system will meet its deadline, providing high-quality (precise) results. In this paper we evaluate by simulation the performance of strategies for the scheduling of parallel jobs (gangs) in a homogeneous distributed real-time system with possible software faults. For each scheduling policy we provide an alternative version which allows imprecise computations. We propose a performance metric applicable to our problem, which takes into account the number of jobs guaranteed, as well as the precision of the results of each guaranteed job. The simulation results show that the alternative versions of the algorithms outperform their respective counterparts. To the best of our knowledge, a real-time gang scheduling approach that utilizes imprecise computations has never been discussed in the literature before.

Keywords: distributed real-time systems, gang scheduling, software faults, imprecise computations, performance evaluation

1. Introduction

Most of today's applications have an inherent need for fault tolerance and high-quality results within precise timing constraints. This has made the use of distributed real-time systems vital in our daily life. Such systems are used for the control of nuclear power plants, financial markets, radar applications and wireless communications.

In real-time systems the correctness of the system does not depend only on the logical results of the computations, but also on the time at which the results are produced. That is, the jobs in a real-time system have *deadlines* which must be met. If a real-time job cannot meet its deadline, then its results will be useless, or even worse, catastrophic for the system and the environment that is under control [1]. Therefore, a real-time system must guarantee that every job will complete its execution before its deadline. Moreover, it must tolerate possible *software faults* that may cause failures during the execution of a job.

Consequently, the most important aspect of a distributed real-time system is the *scheduling algorithm* which is responsible for the allocation of processors to jobs and determines the order in which jobs will be executed on processors. Two commonly used policies for the scheduling of real-time jobs are the *earliest deadline first* (EDF) and the *least laxity first* (LLF) algorithms [2, 3].

In real-time systems a job may not have enough time to complete its execution before its deadline. In that case, the job is considered to be lost, since its execution does not return any results. To confront this problem, Lin et al. [4] have proposed a technique called *imprecise computations*, according to which the execution of a real-time job is allowed to return intermediate (*imprecise*) results of poorer, but still acceptable quality, when the deadline of the job

SIMULATION, Vol. 85, Issue 8, August 2009 525–536
© 2009 The Society for Modeling and Simulation International
DOI: 10.1177/0037549709340729
Figures 3, 4, 7 appear in color online: http://sim.sagepub.com

cannot be met. It is assumed that every job is *monotone*, that is, the accuracy of its intermediate results is increased as more time is spent to produce them. If the execution of a monotone job is fully completed, then the results are the most precise possible [5]. In weather forecasting systems, for example, enhancement routines are used in order to increase the quality of a satellite image. Imprecise computations allow the scheduler to decide whether sufficient time is available to perform all of the enhancement routines. If not, then none or only some of the enhancements are performed. In this way, the system may produce some images of inferior quality, but avoids missing any deadlines. A more critical application which benefits from the utilization of the imprecise computations technique is the collision detection system in aircraft. The collision detection system collects data from the airplane's radar, identifies possible collisions with nearby aircraft and notifies the pilot. Imprecise computations allow the system to ignore distant airplanes when an imminent collision is detected and therefore an immediate notification to the pilot is absolutely necessary [6].

The technique of imprecise computations, combined with *checkpointing*, has also been used as a way to enhance fault tolerance in real-time systems [7, 8]. A common approach is to consider that a monotone job consists of a *mandatory part*, followed by an *optional part*. In order to produce an acceptable result, the mandatory part of the job must be completed. The optional part refines the result produced by the mandatory part. That is, the precision of the results is further increased if the optional part is allowed to be executed longer. The aim is to guarantee that every job will complete *at least* its mandatory part before its deadline. Furthermore, if a software failure occurs during the execution of a job and the job has completed its mandatory part, then there is no need to *rollback* and re-execute the job. In contrast, the job is aborted, and we accept as its result the one produced by its mandatory part (assuming that a checkpoint takes place when a job completes its mandatory part). Rollback is necessary only when the mandatory part of the job is not completed.

In a distributed real-time system, jobs may be either *periodic* or *aperiodic*. Periodic jobs arrive at regular intervals equal to their period, whereas aperiodic jobs have irregular arrival times, which usually follow a Poisson distribution [1, 9, 10]. Real-time jobs usually consist of frequently communicating tasks which can be processed in parallel. An efficient way to schedule dynamic, parallel jobs is *gang scheduling*. Gang scheduling is a combination of time and space sharing. According to this technique, a parallel job is decomposed into tasks that are grouped together into a gang and scheduled and executed simultaneously on different processors. The number of tasks in a gang must be less than or equal to the number of available processors. With gang scheduling, the tasks of a job need to start execution simultaneously, because in this way the risk of a task waiting to communicate with another task that is currently not running is avoided. Without gang scheduling, the synchronization of a job's tasks would require more context switches and thus additional overhead.

Gang scheduling has been studied extensively in the literature [11–19]. Karatza [20] examines the impact of the presence of critical sporadic non-parallel jobs on the performance of the adapted first come first served (AFCFS) and the largest job first served (LJFS) gang scheduling policies, which are used to schedule non-critical parallel jobs in a distributed system. Lee et al. [21] propose a gang scheduling policy based on EDF for the gang scheduling of a mixed workload composed of real-time and non-real-time parallel jobs. Silva et al. [22] investigate the performance of EDF gang scheduling of real-time periodic jobs.

In this paper, we examine the performance of the EDF and LLF algorithms with *application-directed checkpointing* for the gang scheduling of aperiodic parallel jobs in a homogeneous distributed real-time system, where transient software faults (e.g. exceptions) may cause failures during the execution of a job. When a failure occurs, the affected job has to be rolled back to its last checkpoint and restart execution. We also provide an alternative version for each of the algorithms, which allows imprecise computations. Our goal is to guarantee that all jobs (gangs) that arrive in the system will meet their deadlines, providing high-quality (precise) results. We compare the performance of the scheduling policies by simulation, under various workloads, based on the overall system performance metric, which is explained in Section 5.

This paper extends our previous work [23] by examining the version of LLF that is most commonly used in the literature (e.g. [24, 25]), in addition to the modified version of LLF that we propose in [23]. Furthermore, in this paper we assume that the rollback overhead is not constant for all jobs, but that it depends on the particular job that needs to be rolled back. In addition, we evaluate the performance of the scheduling policies for various software failure probabilities.

To our knowledge, a real-time gang scheduling approach that utilizes imprecise computations has never been discussed in the literature before.

The remainder of this paper is organized as follows: Section 2 gives a description of the system under study and the workload model, Section 3 describes the scheduling policies, Section 4 explains how imprecise computations are incorporated in the scheduling strategies and Section 5 presents and analyzes the simulation results. Finally, Section 6 concludes this paper, providing suggestions for further research.

2. System and Workload Model

The target system is considered to be a homogeneous distributed real-time system, consisting of P processors, each serving its own queue (memory). Non-preemptible, aperiodic real-time jobs with hard deadlines arrive in the system in a Poisson stream with rate λ. Upon arrival, each

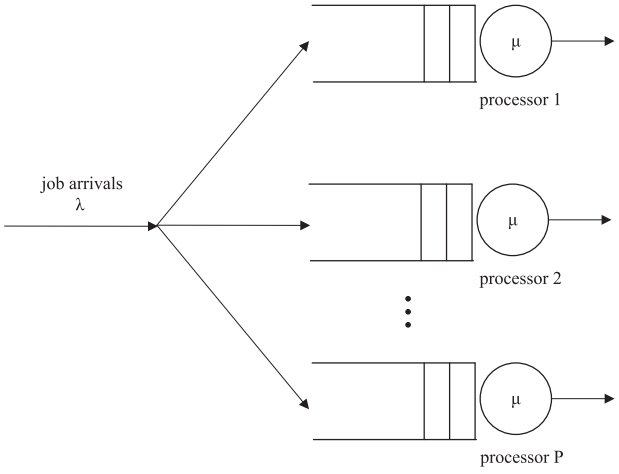


Figure 1. The system model.

job is decomposed by the scheduler into N tasks, which can be processed in parallel. We assume that all tasks in a job require the same service (execution) time. The system model is shown in Figure 1. Hereafter, we use the terms job and gang interchangeably.

The communication between the processors relies solely on message passing. We assume that the processors are fully connected and that the communication is contention-free. Thus, no attention is paid to the routing strategies used for communication. Any communication latencies are assumed to be included implicitly in a job's service time. The service time of a job is exponential with mean $1/\mu$. Every job that arrives in the system is assigned a *priority value* PV according to one of the scheduling policies described in the following section.

Consequently, each job j that arrives in the system is characterized by the following parameters:

- Its (parallel) service time S, which is exponential with mean $1/\mu$.
- Its *absolute deadline* D (measured from time zero).
- Its *relative deadline* D_R, which is the time interval between j's arrival and its absolute deadline D. Here $D_R(j)$ is exponential with mean

$$1/\nu = 2 \times S(j) \tag{1}$$

  and it is greater than or equal to j's service time S.
- Its gang size N, which indicates the required number of processors for j's execution. Here N is an integer uniformly distributed in the closed interval $[1, P]$. Hence, the mean number of tasks per job is equal to $(1 + P)/2$.
- Its priority value PV. The job with the smallest priority value is assumed to have the highest priority.

2.1 Fault Tolerance

A failure may occur during the execution of a job (caused by a transient software fault), with probability FP. FP is the same for all jobs. In this paper we do not examine the case of processor failures, but only the case of software failures. Moreover, we assume that no more than one failure may occur during the execution of a particular job. Each job is responsible for checkpointing its own progress periodically (by saving its intermediate results) at regular intervals during its execution. That is, checkpoints are application directed. The *checkpointing interval* CP of a job j is defined as follows:

$$CP(j) = CPF \times S(j) \tag{2}$$

where CPF is the *checkpointing interval factor*. Here CPF is constant for all jobs and $0 < CPF < 1$. It is assumed that the checkpointing overhead is negligible.

When a failure occurs, we assume that is detected immediately. The interrupted job is rolled back and resumes execution from its last generated checkpoint. The rollback of a job is considered to cause an overhead RB, which depends on the particular job that needs to be rolled back. Specifically, we assume that the *rollback overhead* RB of a job j is equal to

$$RB(j) = RBF(j) \times S(j) \tag{3}$$

where $RBF(j)$ is the *rollback overhead factor* of job j and may be different for each job.

3. Scheduling Policies

In this paper we investigate three algorithms for the scheduling of the gangs: EDF, LLF, and *least initial laxity first* (LILF), a modified version of LLF. In the case of EDF, the priority value of a job j is considered to be its absolute deadline D. That is,

$$PV(j) = D(j). \tag{4}$$

The *laxity* L of a job j at time τ is defined as follows:

$$L_\tau(j) = D(j) - \tau - S_\tau(j) \tag{5}$$

where $S_\tau(j)$ is the remaining service time of j at time τ. That is, the laxity of a job j at time τ is the amount of time left between j's completion and j's deadline, assuming that j could start execution at the current time instant (time τ). In the case of LLF, the priority value of a job j at time τ is considered to be its laxity L:

$$PV_\tau(j) = L_\tau(j). \tag{6}$$

Since $PV_\tau(j)$ changes over time, it must be recalculated at each scheduling step.

At time α of the arrival of a job j, j's laxity L is equal to

$$L_\alpha(j) = D(j) - \alpha - S_\alpha(j). \quad (7)$$

Since $D(j) - \alpha = D_R(j)$ and $S_\alpha(j) = S(j)$, (7) can be rewritten as

$$L_\alpha(j) = D_R(j) - S(j). \quad (8)$$

That is, (8) expresses the laxity of a job at the time of its arrival. Therefore, we define as *initial laxity* IL of a job j the difference between j's relative deadline D_R and j's service time S:

$$IL(j) = D_R(j) - S(j). \quad (9)$$

In the case of LILF, the priority value of a job j is considered to be its initial laxity IL:

$$PV(j) = IL(j). \quad (10)$$

The priority value of a job in the LILF case remains constant over time, as opposed to the priority value of a job in the LLF case, which changes over time. Note that in [23], since we do not use the original version of LLF which we described earlier, but only its modified version (i.e. the LILF algorithm presented in this paper), for simplicity we referred to LILF as LLF.

Consequently, the job that has the highest priority for scheduling, and therefore has each of its tasks placed at the front of its assigned processor's queue, is that which has: (a) the earliest deadline D in the EDF case; (b) the smallest laxity L at the particular time instant in the LLF case; or (c) the smallest initial laxity IL in the LILF case.

3.1 Allocation of Processors

When a job arrives in the system, the scheduler must allocate a processor to each of its constituent tasks. In order for a job to start execution, all of its tasks must be able to start execution simultaneously. Therefore, each task must be sent to a different processor. For the allocation of processors to a job j that has just arrived in the system, the following approach is employed (in all of the above scheduling policies).

We first calculate the *estimated start time* EST that each processor in the system can provide to a task t of j (all of j's tasks are treated as equivalent). If a processor's queue is empty and the processor is idle, then it is provided $EST = 0$. In contrast, if the processor's queue is empty, but the processor is busy executing another task, then its provided EST is equal to the remaining service time of the task in service (including any rollback overhead if the task in service is currently being rolled back). If the processor's queue is not empty, we find the *potential position* of t in the processor's queue, according to j's priority value PV (that is, the position at which t would be placed in the queue if it was assigned to that particular processor). If t would be placed at the head of the queue, then the provided EST of the particular processor is calculated in the same manner as above, depending on the processor's status (i.e. if it is idle or busy). On the other hand, if t would be placed at any other position in the queue, then the processor's provided EST is calculated as follows:

$$EST(t) = EST(t_{\text{previous}}) + S(t_{\text{previous}}) \quad (11)$$

where t_{previous} is the task that would precede t in the particular queue.

When we finish calculating the provided EST of each processor, we allocate processors to j's tasks starting from the one which provides the earliest EST. Subsequently, we select the processor which provides the second earliest EST and so on, until N processors are allocated to j. When we send each task of j to its allocated processor, we place it at its potential position in the processor's queue, as it has been determined earlier. When all tasks of j are at the head of their assigned queues and their respective processors are not occupied by other tasks, then j can start execution.

The deadline of j may be reached while j is in service or waiting for service. In the first case, j is lost and we check its assigned processors' queues to see whether there are any tasks of other jobs that can start execution. If so, they start processing. In the second case, j's tasks are discarded from the queues that are waiting in. If some of j's tasks were waiting at the head of their respective queues, then we check whether the second tasks in those queues belong to gangs that can start execution. If this is the case, they occupy their assigned processors. If j completes its execution before its deadline is reached, we check its assigned processors' queues to see whether there are any tasks such that their gangs can start processing. If so, they start execution.

Clearly, in each processor's queue there may be none, one or more tasks. In the case where there are more than one tasks waiting in a queue, these tasks will belong to different jobs, because, as explained earlier, in gang scheduling all tasks of a job must start execution at the same time. Because of this requirement, some processors may be idle, even with a non-empty queue. Specifically, a task at the head of the queue of an idle processor may be waiting for the other tasks of its gang (which may not be at the head of their assigned queues) to get ready for execution. It is assumed that the scheduling overhead is negligible.

4. Incorporation of Imprecise Computations

For each of the scheduling algorithms analyzed in the previous section, we provide an alternative version (EDF_IC, LLF_IC and LILF_IC), which allows imprecise computations. We assume that every job that arrives in the system is monotone and that it consists of a mandatory part, followed by an optional part.

The mandatory part MP of a job j is considered to be equal to the difference of j's service time and j's checkpointing interval. That is,

$$MP(j) = S(j) - CP(j). \tag{12}$$

Hence, the last generated checkpoint of a job saves the results produced by the job's mandatory part.

The *precision of the results* PR of a completed job j is given by the expression

$$PR(j) = S_{\text{actual}}(j)/S(j) \tag{13}$$

where $S_{\text{actual}}(j)$ is the time that j spent in service. In the case where imprecise computations are not allowed, the precision of the results of a completed job is always $PR = 1$.

Our goal is to guarantee that every job will complete at least its mandatory part before its deadline is reached. In order to achieve this, we employ the notion of *notification time* NT. The notification time of a job j is calculated as follows:

$$NT(j) = D(j) - MP(j). \tag{14}$$

If a job j_1 is waiting for service and its notification time is reached, then we check whether it can start execution: if all of the processors allocated to j_1's tasks are idle or all of the jobs j_i of the tasks in service on those processors have completed their mandatory part, then j_1 can start execution. We abort the jobs j_i (if any) and j_1 occupies its assigned processors. It is important to note that in this case, j_1 will complete only its mandatory part. If we have aborted j_i, then this may result in idle processors in other queues (different from those assigned to j_1). Therefore, we check those queues to see whether there are any tasks such that their gangs can start execution. If this is the case, they occupy their respective processors. On the other hand, if j_1 cannot start execution, it is considered lost, because it will definitely miss its deadline. Its tasks are discarded from the queues that they are waiting in. If some of j_1's tasks were waiting at the head of their respective queues, then we check whether the second tasks in those queues belong to gangs that can start execution. If so, they start processing.

When the deadline of a job that is in service is reached and the job has completed its mandatory part, then it is not considered lost. In contrast, it is aborted and we accept the results it has produced at the time of its deadline. In every other case the job is lost. When a failure occurs during the execution of a job and the job has completed its mandatory part, then there is no need for rollback. In contrast, the job is aborted and we accept the results produced by its mandatory part (i.e. those saved by the job's last checkpoint). Subsequently, we check the assigned processors' queues of the aborted job to see whether there any tasks such that their gangs can start processing. If this is the case, they occupy their allocated processors.

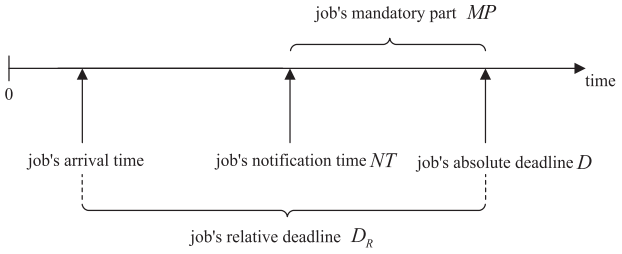


Figure 2. A job's associated times in the imprecise computations case.

In the case where a failure occurs and the affected job has not completed its mandatory part yet, then it is necessary to roll it back and resume its execution from its last checkpoint. We let it execute until it is fully completed or its deadline is reached or it is aborted by a *notified* job (i.e. a job that has reached its notification time). If a failure occurs during the execution of a notified job, then the job is lost and we abort it, because there is no time to roll it back and restart it from its last checkpoint (it will definitely miss its deadline). If there are any tasks in the aborted job's assigned processors' queues that their gangs can start execution, they do so. A job's associated times in the imprecise computations case are shown in Figure 2.

5. Simulation Results and Discussion

For the evaluation of the performance of EDF, LLF, LILF, EDF_IC, LLF_IC and LILF_IC scheduling algorithms in the framework described earlier, we implemented our own discrete-event simulation program in C++ and we performed a series of simulation runs, using the independent replication method. Since EDF, LLF and LILF do not allow imprecise computations, whereas EDF_IC, LLF_IC and LILF_IC do, our performance evaluation needs to take into account not only the number of jobs that meet their deadlines (that is, the number of jobs *guaranteed*), but also the precision of the results of the guaranteed jobs. Therefore, we employ a performance metric called *overall system performance* (OSP), which is defined as follows:

$$OSP = JGR \times ARP \tag{15}$$

where JGR is the *job guarantee ratio* and ARP is the *average result precision*. Here JGR is defined as

$$JGR = \frac{TNJG}{TNJA} \tag{16}$$

where $TNJG$ is the *total number of jobs guaranteed* and $TNJA$ is the *total number of job arrivals*. Here ARP is given by the expression:

$$ARP = \frac{\sum_{i=1}^{TNJG} PR(j_i)}{TNJG}. \tag{17}$$

Hence, (15) can be rewritten as

$$OSP = \frac{\sum_{i=1}^{TNJG} PR(j_i)}{TNJA}. \quad (18)$$

Obviously, OSP is always in the range [0, 1]. In the best case, all of the jobs that arrive in the system are guaranteed and are fully completed and, hence, $OSP = 1$. In the worst case, all of the jobs that arrive in the system are lost and therefore $OSP = 0$. If not all of the jobs in the system are lost, then definitely $ARP > 0$. It is important to note that in the case of EDF, LLF and LILF where imprecise computations are not allowed, the precision of the results of a guaranteed job is always $PR = 1$.

In our experiments, the jobs were generated randomly and we used the following values for the simulation input parameters: the number of processors of the distributed real-time system is $P = 16$, the service rate of the jobs is $\mu = 1$, the gang size N of each job is uniformly distributed in the range [1, 16], the checkpointing interval factor of the jobs is $CPF = 0.25$ (i.e. a job generates a checkpoint when the 25%, 50% and 75% of its service time is completed) and the rollback overhead factor RBF of each job is uniformly distributed in the range [0.15, 0.3].

We investigated the performance of each algorithm under different failure probabilities ($FP = 0, 0.05, 0.1, 0.15$) with respect to the arrival rate λ of the jobs, for $\lambda = 1, 1.25, 1.5, 1.75$. Since each job requires 8.5 processors on average, in order for the system to be stable the arrival rate must satisfy the following condition:

$$8.5\lambda < P\mu \quad (19)$$

and since $P = 16$ and $\mu = 1$, the arrival rate must be

$$\lambda < 1.8824 \quad (20)$$

For each set of input parameters we ran 30 replications of the simulation with different seeds of random numbers and for 10^7 completed (i.e. guaranteed) jobs in each replication. For every mean value, a 95% confidence interval was evaluated. The half-widths of all confidence intervals are less than 5% of their respective mean values. The simulation input parameters are shown in Table 1, whereas the notation used in this paper is summarized in Table 2. We conducted two sets of experiments: one for the case where imprecise computations are not allowed (EDF, LLF and LILF) and one for the imprecise computations case (EDF_IC, LLF_IC and LILF_IC). The simulation results are analyzed below.

5.1 Non-imprecise Computations Case

We first compared the performance of the EDF, LLF and LILF algorithms (based on the OSP metric) under different failure probabilities, with respect to the arrival rate λ. Figure 3 demonstrates this comparison. It is observed that

Table 1. Simulation input parameters.

Parameter	Value
P	16
Scheduling algorithm	EDF, LLF, LILF, EDF_IC, LLF_IC, LILF_IC
Required number of guaranteed jobs	10^7
λ	1, 1.25, 1.5, 1.75
μ	1
N	$\sim U$ [1–16]
FP	0, 0.05, 0.1, 0.15
CPF	0.25
RBF	$\sim U$ [0.15, 0.3]

Table 2. List of notation.

Notation	Definition
P	Number of processors
λ	Arrival rate of the jobs
μ	Service rate of the jobs
S	Service (execution) time of a job
D	Absolute deadline of a job
D_R	Relative deadline of a job
L	Laxity of a job
IL	Initial laxity of a job
N	Gang size of a job
PV	Priority value of a job
FP	Failure probability
CP	Checkpointing interval of a job
CPF	Checkpointing interval factor of the jobs
RB	Rollback overhead of a job
RBF	Rollback overhead factor of a job
EDF	Earliest deadline first job scheduling algorithm
EDF_IC	Alternative version of EDF that allows imprecise computations
LLF	Least laxity first job scheduling algorithm
LLF_IC	Alternative version of LLF that allows imprecise computations
LILF	Least initial laxity first job scheduling algorithm
LILF_IC	Alternative version of LILF that allows imprecise computations
EST	Estimated start time of a job
MP	Mandatory part of a job (applicable to the imprecise computations case)
NT	Notification time of a job (applicable to the imprecise computations case)
PR	Precision of results of a completed job
OSP	Overall system performance
JGR	Job guarantee ratio
ARP	Average result precision
$TNJG$	Total number of jobs guaranteed
$TNJA$	Total number of job arrivals

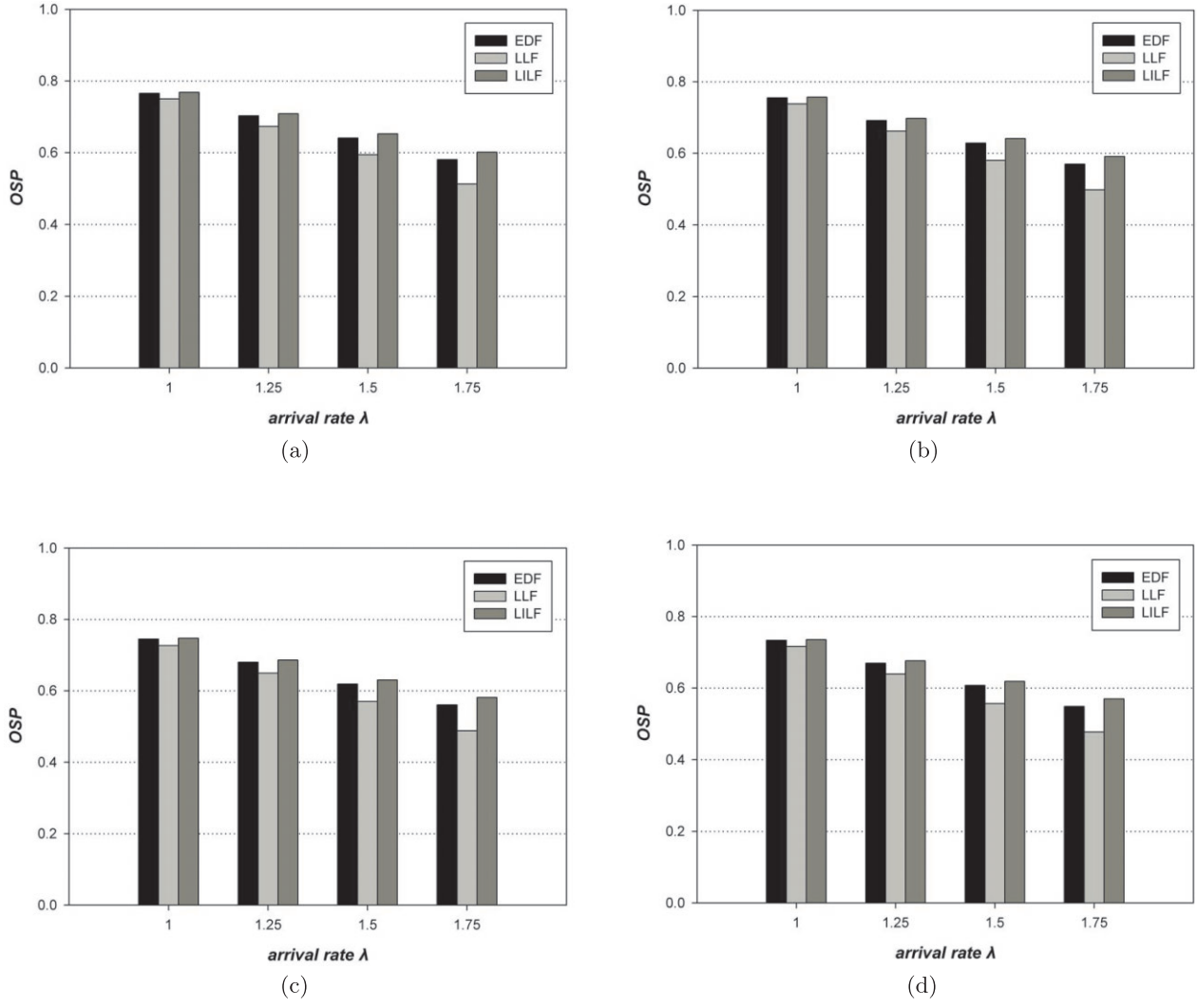


Figure 3. Plot of OSP versus the arrival rate λ for EDF, LLF and LILF for: (a) $FP = 0$; (b) $FP = 0.05$; (c) $FP = 0.1$; and (d) $FP = 0.15$.

LILF performs better than EDF and LLF, particularly under heavy workload and high failure probability. LLF exhibits the poorest performance between the three policies. Specifically, the overall system performance under failure probability $FP = 0.15$ is $0.5485 \leq OSP \leq 0.7337$ in the case of EDF, $0.4769 \leq OSP \leq 0.7165$ in the case of LLF and $0.5696 \leq OSP \leq 0.7354$ in the case of LILF. As the arrival rate of the jobs in the system increases, the OSP of all algorithms decreases. This is due to the higher waiting time of the jobs in the queues, which causes more jobs to miss their deadlines. Furthermore, EDF, LLF and LILF perform worse under high failure probabilities. As the number of software failures increases, more jobs need to be rolled back and resume execution. Therefore, a job that is affected by a software failure requires extra processing time and, thus, it may reach its deadline before its completion.

Since imprecise computations are not allowed, the average result precision $ARP = 1$ for all strategies. Therefore, the performance of EDF, LLF and LILF depends only on the job guarantee ratio JGR parameter. Thus, in this case both OSP and JGR express the job guarantee ratio, that is the probability of not missing a job's deadline.

5.2 Imprecise Computations Case

Subsequently, we compared the performance of the EDF_IC, LLF_IC and LILF_IC policies (Figure 4). In this case, EDF_IC outperforms the other two algorithms, while LLF_IC exhibits the poorest overall system performance. The difference in the performance of the three policies increases under heavy workload and high failure probability. Specifically, the overall system performance

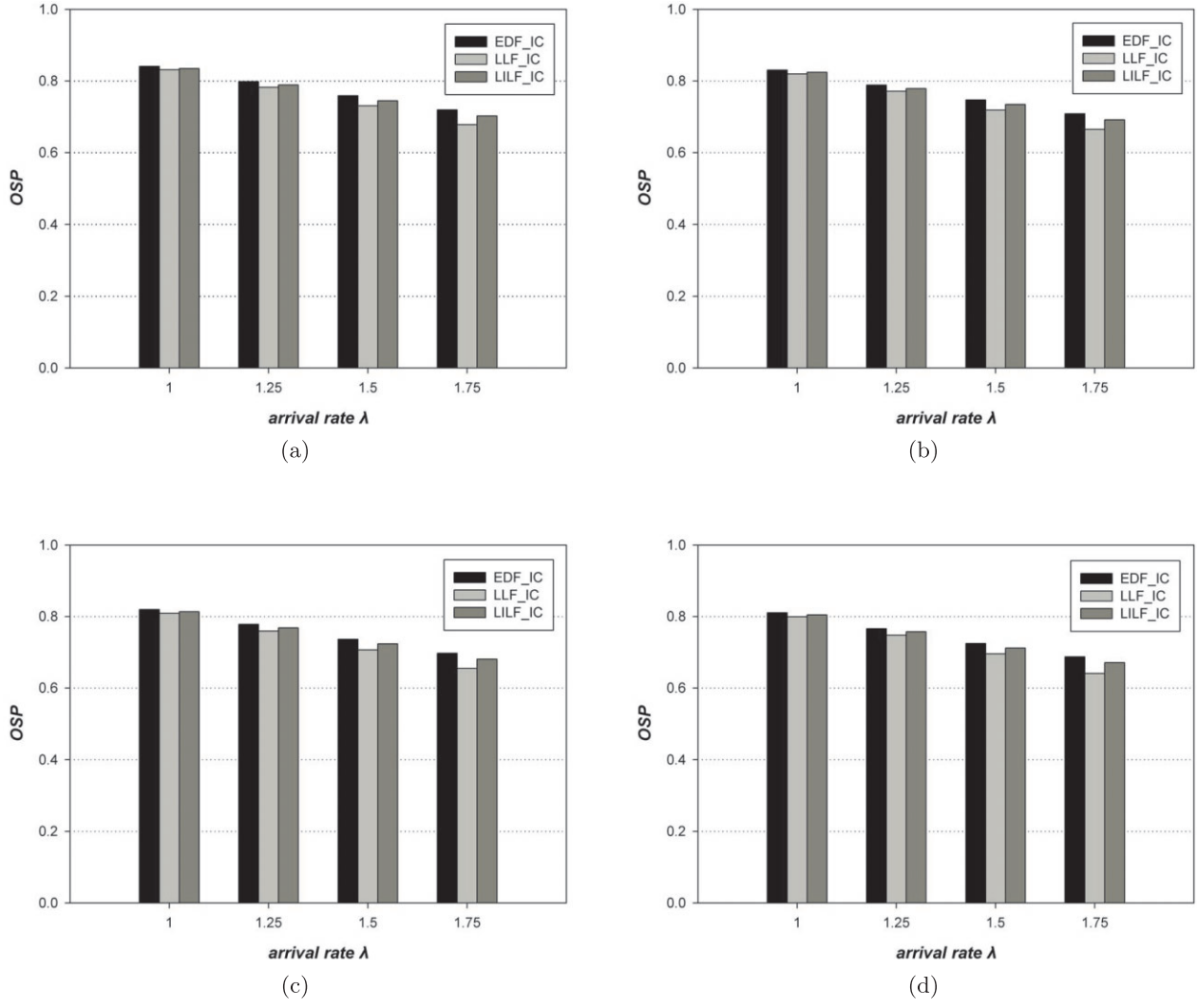


Figure 4. Plot of OSP versus the arrival rate λ for EDF_IC, LLF_IC and LILF_IC for: (a) $FP = 0$; (b) $FP = 0.05$; (c) $FP = 0.1$; and (d) $FP = 0.15$.

under failure probability $FP = 0.15$ is $0.6871 \leq OSP \leq 0.8104$ in the case of EDF_IC, $0.6407 \leq OSP \leq 0.7992$ in the case of LLF_IC and $0.6708 \leq OSP \leq 0.8045$ in the case of LILF_IC. As in the case where imprecise computations are not allowed, the overall system performance of the three strategies decreases as the workload and the failure probability increase. In this case however, since imprecise computations are allowed, the performance of the algorithms depends on both of the parameters JGR (Figure 5) and ARP (Figure 6).

In terms of the JGR parameter, EDF_IC performs better than LLF_IC and LILF_IC. However, in terms of the ARP parameter, LILF_IC provides results of higher precision than the EDF_IC and LLF_IC algorithms. Since both JGR and ARP have the same weight in the overall system performance, the difference in the job guarantee ratio of the three policies dominates, because it is greater than the difference in their average result precision. Therefore, EDF_IC performs better than LLF_IC and LILF_IC in terms of the overall system performance metric.

Figure 7 illustrates the comparison between the original versions and the imprecise computations counterparts of EDF, LLF and LILF. In the case of EDF_IC, the improvement in the overall system performance under heavy workload and high failure probability is 25.27%, compared with EDF. In the case of LLF_IC the improvement is 34.35% compared with LLF, whereas in the case of LILF_IC the improvement is 17.77% compared with LILF. The LLF policy benefits more from the incorporation of imprecise computations than EDF and LILF do. However, it still performs worse than the other two policies in the imprecise computations case. Our modified

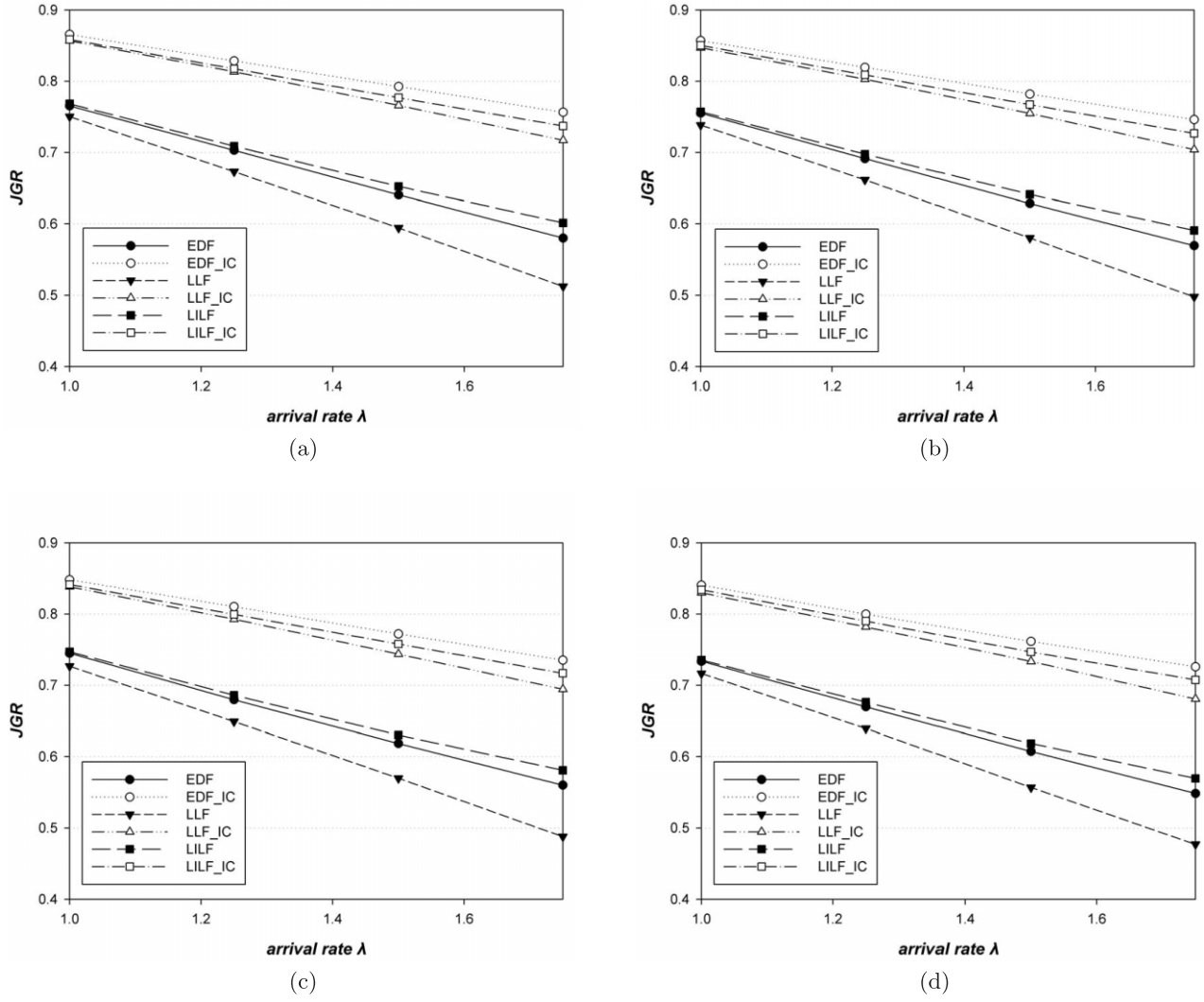


Figure 5. Plot of JGR versus arrival rate λ for EDF, EDF_IC, LLF, LLF_IC, LILF and LILF_IC for: (a) $FP = 0$; (b) $FP = 0.05$; (c) $FP = 0.1$; and (d) $FP = 0.15$.

version of LLF, i.e. LILF, performs better than LLF, regardless of whether imprecise computations are allowed or not.

Clearly, EDF_IC outperforms all of the other algorithms, giving overall system performance $OSP = 0.8407$ in the best case ($\lambda = 1, FP = 0$) and $OSP = 0.6871$ in the worst case ($\lambda = 1.75, FP = 0.15$). Comparing the best performing policy that does not allow imprecise computations (i.e. LILF) and the best performing policy that allows imprecise computations (i.e. EDF_IC), the improvement in the case of EDF_IC is remarkable. Specifically, in the case of EDF_IC the overall system performance under heavy workload and high failure probability is increased by 20.63%, compared with LILF.

In terms of job guarantee ratio JGR (Figure 5), under heavy workload and high failure probability, the improvement in the case of EDF_IC is 32.36% compared with EDF, in the case of LLF_IC the improvement is 42.73% compared with LLF, whereas in the case of LILF_IC the improvement is 24.19% compared with LILF. In EDF_IC, LLF_IC and LILF_IC the average result precision ARP is always approximately equal to one (Figure 6). Specifically, when the probability of a failure during the execution of a job is $FP = 0.15$, the average result precision is $0.9465 \leq ARP \leq 0.9642$ in the case of EDF_IC, $0.9412 \leq ARP \leq 0.9631$ in the case of LLF_IC and $0.9482 \leq ARP \leq 0.9647$ in the case of LILF_IC. Therefore, the loss of result precision is not significant, since under heavy workload and high failure probability it is 5.35%, 5.88% and 5.18% for EDF_IC, LLF_IC and LILF_IC, respectively. Thus, the tradeoff between average result precision and job

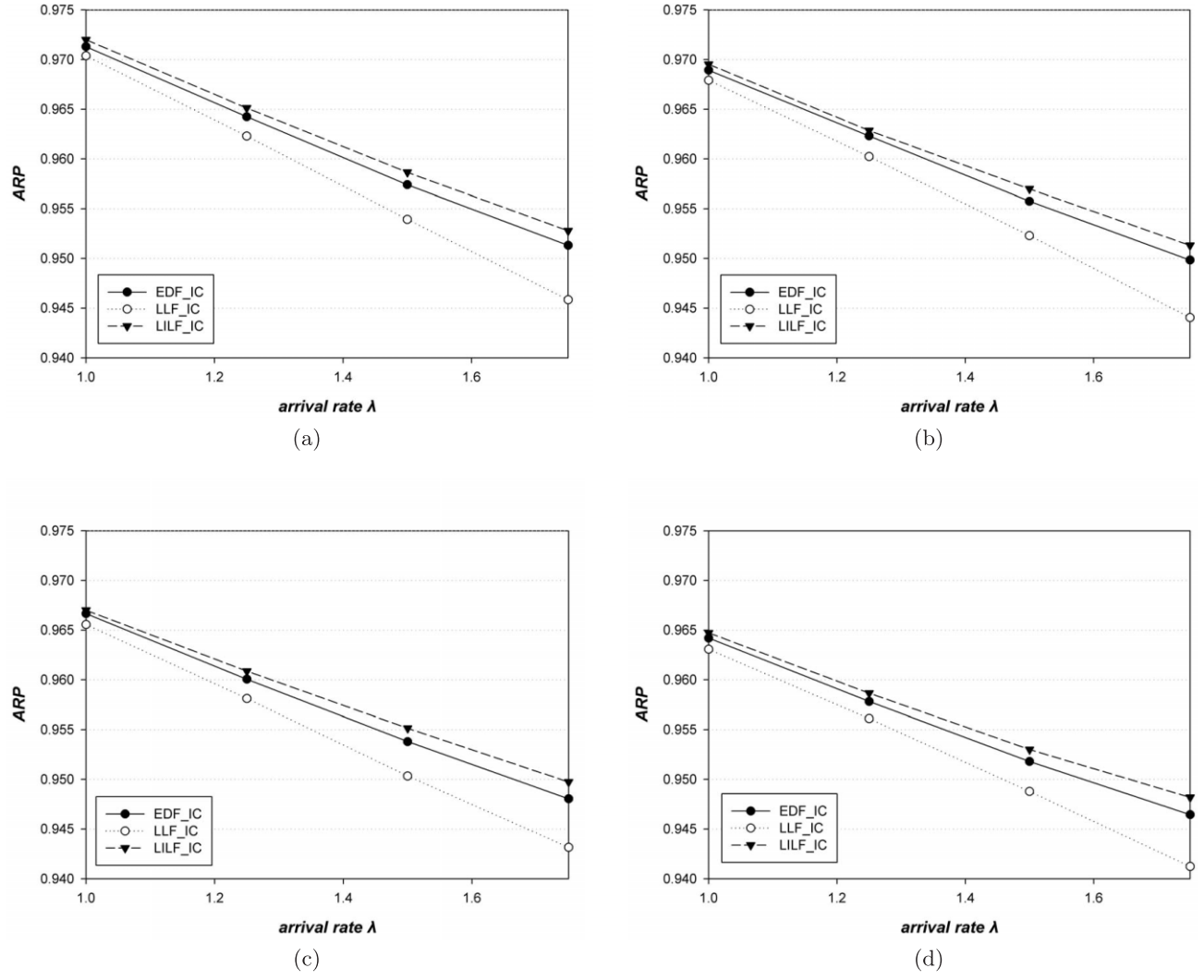


Figure 6. Plot of ARP versus arrival rate λ for EDF_IC, LLF_IC and LILF_IC for: (a) $FP = 0$; (b) $FP = 0.05$; (c) $FP = 0.1$; and (d) $FP = 0.15$.

guarantee ratio ultimately gives a higher overall system performance.

6. Conclusions and Future Work

In this paper we have evaluated the performance of the EDF, LLF and LILF (a modified version of LLF) policies with application-directed checkpointing for the gang scheduling of non-preemptible, aperiodic parallel jobs in a homogeneous distributed real-time system, where software failures may occur during the execution of a job. For each algorithm we have provided an alternative version, which allows imprecise computations (EDF_IC, LLF_IC and LILF_IC).

The performance of the algorithms, in terms of the overall system performance OSP metric, was examined by simulation, under various workloads and different failure probabilities. Our simulation experiments show that in this case study EDF_IC, LLF_IC and LILF_IC perform better than EDF, LLF and LILF, respectively. Moreover, EDF_IC exhibits promising performance and outperforms all of the other policies.

However, in this paper we did not take into account the scheduling overhead, nor the case where the jobs are preemptible. Therefore, further research is needed in order to address these problems. In addition, we could examine cases where the interarrival and service times, as well as the relative deadlines of the jobs, exhibit high variability.

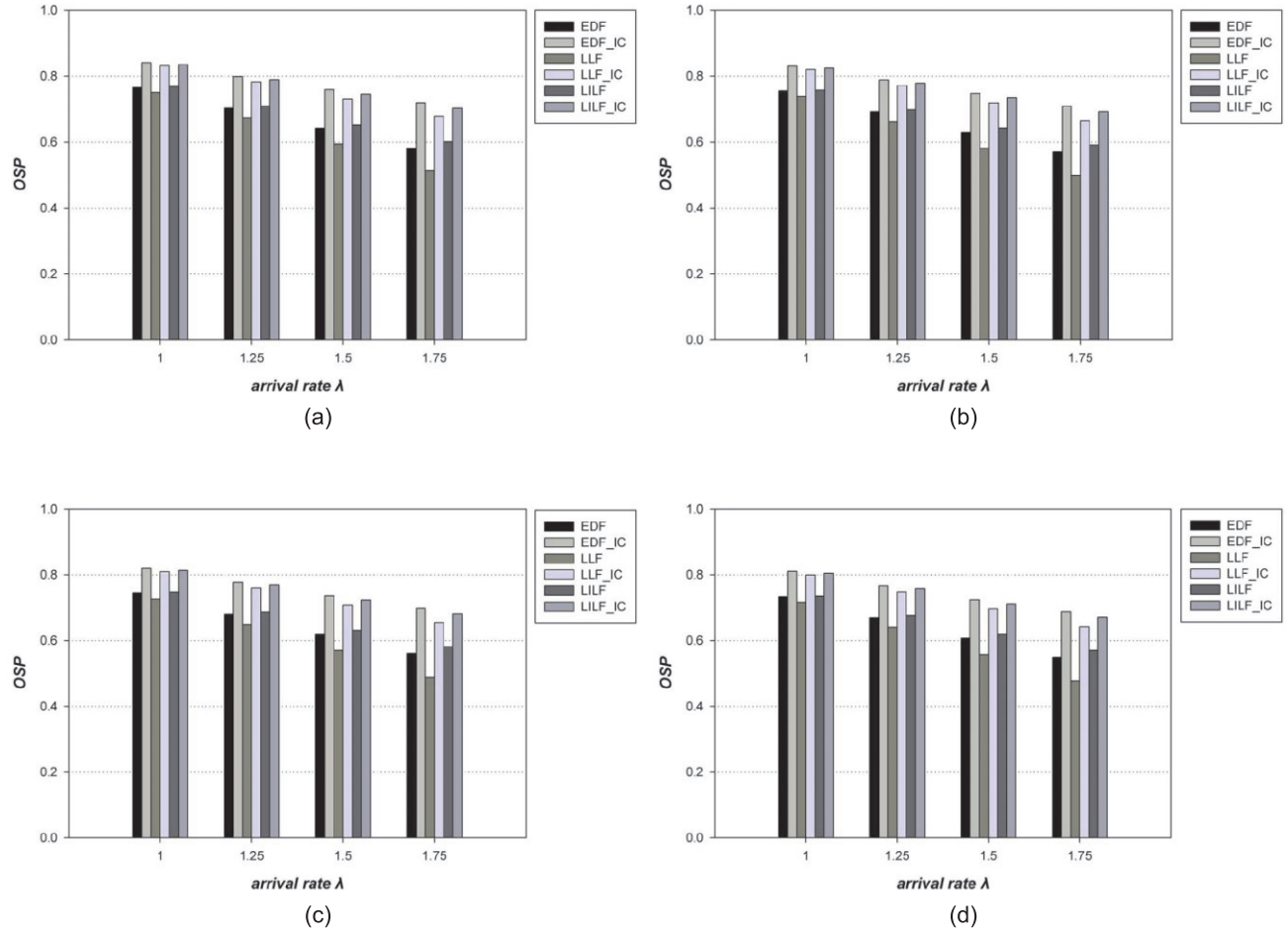


Figure 7. Plot of OSP versus the arrival rate λ for EDF, EDF_IC, LLF, LLF_IC, LILF and LILF_IC for: (a) $FP = 0$; (b) $FP = 0.05$; (c) $FP = 0.1$; and (d) $FP = 0.15$.

7. References

[1] Buttazzo, G.C. 2004. *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*, 2nd edition, Springer, Berlin.

[2] Liu, C.L. and J.W. Layland. 1973. Scheduling algorithms for multiprogramming in a hard real-time environment. *Journal of the ACM*, 20(1): 46–61.

[3] Mok, A.K. 1983. *Fundamental Design Problems of Distributed Systems for the Hard Real-time Environment*, PhD Thesis. Massachusetts Institute of Technology.

[4] Lin, K.J., S. Natarajan and J.W.S. Liu. 1987. Imprecise results: utilizing partial computations in real-time systems. In *Proceedings of the 8th IEEE Real-Time Systems Symposium (RTSS'87)*, San Jose, CA, pp. 210–217.

[5] Liu, J.W.S., K.J. Lin, W.K. Shih, A.C. Yu, J.Y. Chung and W. Zhao. 1991. Algorithms for scheduling imprecise computations. *IEEE Computer*, 24(5): 58–68.

[6] Hull, D.L. 2000. *An Environment for Imprecise Computations*, PhD Thesis, University of Illinois at Urbana–Champaign.

[7] Han, C.C., K.G. Shin and J. Wu. 2003. A fault-tolerant scheduling algorithm for real-time periodic tasks with possible software faults. *IEEE Transactions on Computers*, 52(3): 362–372.

[8] Liu, J.W.S., W.K. Shih, K.J. Lin, R. Bettati and J.Y. Chung. 1994. Imprecise computations. *Proceedings of the IEEE*, 82(1): 83–94.

[9] Harchol–Balter, M., M. Crovella and C.D. Murta. 1998. On choosing a task assignment policy for a distributed server system. In *Proceedings of the 10th International Conference on Computer Performance Evaluation: Modelling Techniques and Tools* (*Lecture Notes in Computer Science*, Vol. 1469), Springer, Berlin, pp. 231–242.

[10] Law, A.M. and W.D. Kelton. 2000. *Simulation Modeling and Analysis*, 3rd edition, McGraw-Hill, New York.

[11] Feitelson, D.G. and M.A. Jette. 1997. Improved utilisation and responsiveness with gang scheduling. *Job Scheduling Strategies for Parallel Processing* (*Lecture Notes in Computer Science*, Vol. 1291), Springer, Berlin, pp. 23–26.

[12] Karatza, H.D. 1999. A simulation-based performance analysis of gang scheduling in a distributed system. In *Proceedings of the 32nd Annual Simulation Symposium*, San Diego, CA, pp. 26–33.

[13] Karatza, H.D. 1999. Gang scheduling in a distributed system with processor failures. In *Proceedings of the UK Performance Engineering Workshop*, Bristol, UK, pp. 199–208.

[14] Karatza, H.D. 2000. Gang scheduling and I/O scheduling in a multiprocessor system. In *Proceedings of the 2000 International Symposium on Performance Evaluation of Computer*

and Telecommunication Systems (SPECTS 2000), Vancouver, Canada, pp. 245–252.

[15] Karatza, H.D. 2003. Gang scheduling in a distributed system under processor failures and time-varying gang size. In *Proceedings of the 9th IEEE Workshop on Future Trends of Distributed Computing Systems (FTDCS'03)*, San Juan, Puerto Rico, pp. 330–336.

[16] Karatza, H.D. 2006. Scheduling gangs in a distributed system. *International Journal of Simulation: Systems, Science and Technology*, 7(1): 15–22.

[17] Karatza, H.D. 2006. Performance analysis of gang scheduling in a partitionable parallel system. In *Proceedings of the 20th European Conference on Modelling and Simulation*, Sankt Augustin, Germany, pp. 699–704.

[18] Karatza, H.D. 2007. Performance of gang scheduling policies in the presence of critical sporadic jobs in distributed systems. In *Proceedings of the 2007 International Symposium on Performance Evaluation of Computer and Telecommunication Systems (SPECTS'07)*, San Diego, CA, pp. 547–554.

[19] Zhang, Y., A. Yang, A. Sivasubramaniam and J. Moreira. 2003. Gang scheduling extensions for I/O intensive workloads. *Job Scheduling Strategies for Parallel Processing* (*Lecture Notes in Computer Science*, Vol. 2862), Springer, Berlin, pp. 83–207.

[20] Karatza, H.D. 2008. The impact of critical sporadic jobs on gang scheduling performance in distributed systems. *Simulation: Transactions of the Society for Modeling and Simulation International*, 84(2–3): 89–102.

[21] Lee, J.K., C.D. Lin, Y.W. Chang and W.K. Shih. 1998. Real-time gang schedulings with workload models for parallel computers. In *Proceedings of the 1998 IEEE International Conference on Parallel and Distributed Systems (ICPADS'98)*, Tainan, Taiwan, pp. 114–121.

[22] Silva, F.A.B., E.P. Lopes, E.P.L. Aude, F. Mendes, J.T.C. Silveira, H. Serdeira, M. Martins and W. Cirne. 2002. Response time analysis of gang scheduling for real time systems. In *Proceedings of the 2002 International Symposium on Performance Evaluation of Computer and Telecommunication Systems (SPECTS'02)*, San Diego, CA, pp. 297–301.

[23] Stavrinides, G.L. and H.D. Karatza. 2008. Performance evaluation of gang scheduling in distributed real-time systems with possible software faults. In *Proceedings of the 2008 International Symposium on Performance Evaluation of Computer and Telecommunication Systems (SPECTS'08)*, Edinburgh, UK, pp. 1–7.

[24] Cottet, F., J. Delacroix, C. Kaiser and Z. Mammeri. 2002. *Scheduling in Real-Time Systems*, John Wiley & Sons, Inc., New York.

[25] Sha, L., T. Abdelzaher, K.E. Arzen, A. Cervin, T. Baker, A. Burns, G. Buttazzo, M. Caccamo, J. Lehoczky and A.K. Mok. 2004. Real time scheduling theory: a historical perspective. *Real-Time Systems*, 28(2–3): 101–155.

Georgios L. Stavrinides *is a PhD student in the Department of Informatics, Aristotle University of Thessaloniki, Greece. His research interests include performance evaluation of distributed real-time systems, simulation and scheduling algorithms.*

Helen D. Karatza *is an Associate Professor in the Department of Informatics, Aristotle University of Thessaloniki, Greece. Her research interests include performance evaluation of parallel and distributed systems, scheduling and simulation.*